

MediQ — App Development Spec

Source: *GROUP 1 – Event-Driven Programming Docs* (52 pp., Liceo de Cagayan University, Aug 2026). Extracted for developers.

1. Overview

- **App:** Mobile appointment and consultation scheduling for **one** private hospital (Cagayan de Oro).
- **Platform:** Android.
- **Methodology:** RAD (Requirements Planning → User Design → Construction → Cutover).
- **Roles:** Patient, Doctor, Secretary.
- **Core rule:** Real-time time-slot management. A booked slot becomes unavailable to everyone else; double booking must be impossible.
- **SDGs:** 3 (Good Health), 9 (Industry, Innovation, Infrastructure).

2. Tech Stack

Doc does not name a stack. Team's chosen stack:

- Mobile: Kotlin + Jetpack Compose (Android Studio)
- Backend: Node.js (Express)
- DB: MySQL
- APIs: Google Calendar, Google Maps
- Design: Figma
- VCS: GitHub

3. Features by Role

Patient

- Register / log in
- Search doctors by **specialty** and **clinic location**
- View doctor profile, consultation schedule, available slots
- Book, cancel, or request reschedule
- Receive confirmation and reminder notifications
- Message the **secretary** (appointment concerns only)

Doctor

- Manage consultation schedule
- View patient appointments

- Update appointment status
- Block unavailable schedules

Secretary

- Manage doctor info
- Monitor appointment records and live slot availability
- Confirm or reschedule appointments; review reschedule/cancel requests
- Answer patient messages
- Generate basic appointment reports

4. Out of Scope

EMR, online payments, telemedicine/video, e-prescriptions, lab, pharmacy, inventory, billing. Patient↔Doctor direct messaging is **not** supported (secretary only). Messaging must not give diagnosis, treatment advice, or triage.

5. Architecture

Components (from Figure 4):

- **MediQ Application System** with modules: Authentication & Role Management, Appointment & Schedule Management, Appointment Status Management, Messaging & Notifications
- **MediQ Database** (store / retrieve)
- **Notification Service** → sends Confirmation / Reminder to Patient
- Patient, Doctor, Secretary each send Request / get Response

6. Database Schema (Data Dictionary)

Table	Fields
Users	UserID PK AI, Role VARCHAR(20) NN (Patient/Doctor/Secretary), Email VARCHAR(100) NN UNIQUE, PasswordHash VARCHAR(255) NN, ContactNumber VARCHAR(15) NN, DateCreated DATETIME default now
Patients	PatientID PK, UserID FK→Users, FullName VARCHAR(100) NN, DateOfBirth DATE NN, Gender VARCHAR(10) NN, Address VARCHAR(150) null
Doctors	DoctorID PK, UserID FK, SpecialtyID FK→Specialties, FullName VARCHAR(100) NN, ClinicLocation VARCHAR(150) NN, LicenseNumber VARCHAR(30) UNIQUE
Secretaries	SecretaryID PK, UserID FK, FullName VARCHAR(100) NN
Specialties	SpecialtyID PK, SpecialtyName VARCHAR(50) NN
Schedules	ScheduleID PK, DoctorID FK, DayOfWeek VARCHAR(10) NN, StartTime TIME NN, EndTime TIME NN, Status ENUM(Active, Inactive)
Timeslots	SlotID PK, ScheduleID FK, SlotDate DATE NN, SlotTime TIME NN, Status ENUM(Available, Booked, Blocked)
Appointments	AppointmentID PK, PatientID FK, DoctorID FK, SlotID FK (1:1), Status ENUM(Pending, Confirmed, Completed, Cancelled), ReasonForVisit VARCHAR(200) NN, DateCreated DATETIME default now
Messages	MessageID PK, PatientID FK, SecretaryID FK, SenderType (Patient/Secretary), MessageText TEXT, Timestamp DATETIME default now
Notifications	NotificationID PK, UserID FK, AppointmentID FK, Type ENUM(Reminder, Confirmation, Cancellation), Content VARCHAR(200), IsRead BOOLEAN default false

Relationships: Users 1-1 Patients/Doctors/Secretaries; Specialties 1-N Doctors; Doctors 1-N Schedules; Schedules 1-N Timeslots; Timeslots 1-1 Appointments; Patients 1-N Appointments; Patients/Secretaries N-N via Messages; Users 1-N Notifications.

7. Booking Logic (critical)

Validate + reserve + create appointment in **one DB transaction** using `(SELECT ... FOR UPDATE)`.

sql

```
START TRANSACTION;
SELECT Status FROM Timeslots WHERE SlotID = ? FOR UPDATE;
-- if Status != 'Available' → ROLLBACK, return "slot unavailable"
UPDATE Timeslots SET Status = 'Booked' WHERE SlotID = ?;
INSERT INTO Appointments (PatientID, DoctorID, SlotID, Status, ReasonForVisit)
VALUES (?, ?, ?, 'Pending', ?);
COMMIT;
```

Add `UNIQUE(SlotID)` on Appointments as a second guard. After commit: create Notification; sync Calendar async.

8. Activity Flow (Figure: Activity Diagram)

1. Patient: Log in → search doctor / view schedule → select slot → submit booking.
2. System: validate slot. Unavailable → show message, back to slot selection. Available → create appointment, mark Booked, send confirmation + reminder.
3. Change appointment?
 - **No:** receive reminder, end.
 - **Yes:** request cancel or reschedule.
 - **Cancel:** cancel, release slot, send cancellation notice.
 - **Reschedule:** secretary reviews → new slot available? Yes: approve, update appointment + slot, send new confirmation. No: notify patient slot unavailable.
4. Doctor: manage consultation schedule. Secretary: review request, coordinate appointment.

9. Event-Driven Design

Event	Handler response
Tap Book	Validate slot → reserve or reject → update UI with message
Appointment saved	Data-change event → refresh slot availability
Scheduled time reached	Fire confirmation/reminder notification
App resume (lifecycle)	Refresh schedule to avoid stale data
Cancel tap	Release slot, notify
Reschedule request	Notify secretary; on approval update slot + notify patient
New message	Update secretary dashboard / patient thread

Every event needs a defined expected response and a test case.

10. Non-Functional Requirements

- **Security:** role-based access control, password hashing, secure transmission (HTTPS), input validation, session management, audit logging.
- **Privacy:** collect only what scheduling needs; retention/deletion per hospital policy; comply with RA 10173 (Data Privacy Act 2012). Treat names, contacts, schedules, messages as sensitive.
- **Usability:** plain language, few steps, show live availability, complete doctor profiles and clinic locations, clear appointment statuses. Don't assume all patients are digitally confident.
- **Reliability:** accurate schedules, understandable errors, exception handling.
- **Quality model:** ISO/IEC 25010 — Functional Suitability, Usability, Reliability, Performance Efficiency, Security.

11. Functional Test Cases (minimum)

- Book available slot → success, slot Booked, notification created
- Book already-taken slot → rejected with message
- Two simultaneous bookings on one slot → exactly one succeeds
- Cancel → slot released, cancellation notice
- Reschedule request → secretary approve/reject paths
- Reminder/confirmation notification creation

- Doctor blocks slot → not bookable
- Role access: patient cannot open doctor/secretary functions
- App resume refreshes slots

12. Inconsistencies in the Doc (resolve before building)

1. **Status values:** Lit review lists Available, Booked, Cancelled, Reschedule Requested, Completed, No-show; data dictionary Appointments has only Pending, Confirmed, Completed, Cancelled. Add `Reschedule Requested` (needed by the flow) and decide on `No-show`.
2. **Slot status:** Proposed System says Available → Reserved/Unavailable; dictionary says Available/Booked/Blocked. Use dictionary values.
3. **Quality criteria:** Ch.1 and 3 (Evaluation) use 5 ISO criteria; Research Instrument lists Functionality, Reliability, Usability, Efficiency, Maintainability, Portability. Align to one set.
4. **Notification types** lack `Reschedule`; add if reschedule confirmations need distinct handling.
5. **Users.ContactNumber / DateCreated** constraint text is garbled ("CURRENT_DATE" default on a DATETIME); use `CURRENT_TIMESTAMP`.
6. **Missing tables:** no Reschedule Request record and no audit log despite the security requirement.
7. Evaluation is **pending**; interview with the hospital not yet done.

13. Suggested Build Order

1. DB schema + seed (specialties, sample doctors)
2. Auth + role middleware
3. Doctor search + schedules + slot generation
4. Booking transaction + cancel
5. Notifications (confirm/reminder/cancel)
6. Reschedule request flow (secretary)
7. Messaging (patient ↔ secretary)
8. Doctor schedule/block tools, secretary dashboard + reports
9. Concurrency, role-access, and functional tests